PART A: Concept Application

```
import random
import string

special_chars = "!@#$%^&*"


def analyze_password(password):
    score = 0
    missing = []

    length = len(password)

    # Length scoring
    if length >= 16:
        score += 3
    elif length >= 12:
        score += 2
    elif length >= 8:
        score += 1
    else:
        missing.append("too short")

    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False

    repeat_count = 1
    no_repeats = True

    # Character inspection
    for i, char in enumerate(password):

        if char.isupper():
            has_upper = True
        elif char.islower():
            has_lower = True
        elif char.isdigit():
            has_digit = True
        elif char in special_chars:
            has_special = True

        # Check repetition
        if i > 0 and char == password[i - 1]:
            repeat_count += 1
            if repeat_count > 2:
                no_repeats = False
        else:
            repeat_count = 1

    # Score character types
    if has_upper:
        score += 1
    else:
        missing.append("uppercase")

    if has_lower:
        score += 1
    else:
        missing.append("lowercase")

    if has_digit:
        score += 1
    else:
        missing.append("digit")

    if has_special:
        score += 1
    else:
        missing.append("special char")

    if no_repeats:
        score += 1
    else:
        missing.append("too many repeated characters")

    return score, missing
```

```
def rating(score):
    if score <= 2:
        return "Weak"
    elif score <= 4:
        return "Medium"
    elif score <= 6:
        return "Strong"
    else:
        return "Very Strong"


def generate_password(length):
    chars = string.ascii_letters + string.digits + string.punctuation
    password = ""

    for _ in range(length):
        password += random.choice(chars)

    return password


# --- Password Analyzer Loop ---

while True:
    password = input("Enter password: ")

    score, missing = analyze_password(password)

    print(f">> Strength: {score}/7 ({rating(score)})")

    if missing:
        print(">> Missing:", ", ".join(missing))

    if score >= 5:
        print(">> Password accepted!")
        break
    else:
        print(">> Try again...\n")


# --- Password Generator ---

length = int(input("\nEnter length for generated password: "))

generated = generate_password(length)

print("\nGenerated Password:", generated)

score, missing = analyze_password(generated)

print(f"Strength: {score}/7 ({rating(score)})")
```

```
Enter password: hello
>> Strength: 2/7 (Weak)
>> Missing: too short, uppercase, digit, special char
>> Try again...

Enter password: H3llo@World!
>> Strength: 7/7 (Very Strong)
>> Password accepted!

Enter length for generated password: 23

Generated Password: 3Qh^UKCzufH@;g,)5qjW5<o
Strength: 8/7 (Very Strong)
```

PART B — Interview Ready

Difference between break and continue (with example)

Feature	`break`	`continue`
Purpose	Terminates the loop immediately	Skips the current iteration
Loop behavior	Control exits the loop	Control moves to the next iteration
Remaining iterations	Not executed	Still executed

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

```
0
1
2
```

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

```
0
1
2
4
```

What is the else clause in for / while loops?

```
Python loops can have an else block that runs only if the loop finishes normally.

The else block does NOT execute if the loop exits via break.
```

When does it execute? Practical use case (hint: search pattern)

The else block runs only when the loop ends without encountering break.

Practical Use Case — Searching

When searching through a list:

```
If the item is found → break

If not found → else executes
```

```
numbers = [1, 3, 5, 7]

for item in numbers:
    if item == 4:
        print("Found 4")
        break
else:
    print("4 not found")
```

```
4 not found
```

```
def find_pairs(numbers, target):
    pairs = []

    for i in range(len(numbers)):
        for j in range(i + 1, len(numbers)):
            if numbers[i] + numbers[j] == target:
                pairs.append((numbers[i], numbers[j]))

    return pairs
```

```
print(find_pairs([1,2,3,4,5], 6))
```

```
[(1, 5), (2, 4)]
```

Optimized Code

```
def find_pairs(numbers, target):
    seen = set()
    pairs = []

    for num in numbers:
        complement = target - num

        if complement in seen:
            pairs.append((complement, num))

        seen.add(num)

    return pairs
```

```
print(find_pairs([1,2,3,4,5], 6))
```

```
[(2, 4), (1, 5)]
```

Debug & Analyze

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, n):  # Bug?
        if n % i == 0:
            return False
    return True
```

Issue 1 — Inefficient Divisor Check

```
for i in range(2, n)

checks every number from 2 up to n-1
```

Issue 2 - Performance Issue

The loop runs:

```
n - 2 iterations
```

Time complexity:

```
O(n)
```

For large numbers (e.g., millions), this becomes inefficient.

```
print(is_prime(7))
print(is_prime(10))
print(is_prime(29))
```

```
True
False
True
```

PART C: AI-Augmented Task

Write a Python program that prints a diamond pattern of asterisks. The user inputs the number of rows for the upper half. Include proper spacing and use nested loops only (no string multiplication tricks).

```
n = int(input("Enter number of rows: "))

# Upper half
for i in range(1, n + 1):
    for j in range(n - i):
        print(" ", end="")
    for k in range(2 * i - 1):
        print("*", end="")
    print()

# Lower half
for i in range(n - 1, 0, -1):
    for j in range(n - i):
        print(" ", end="")
    for k in range(2 * i - 1):
        print("*", end="")
    print()
```

```
Enter number of rows: 2
 *
***
 *
```

Brief Summary — Critical Evaluation

1. Spacing Correctness The program correctly centers the diamond using:

- spaces = `n - i`

- stars = `2*i - 1` This produces a symmetric diamond.

2. Readability The code is understandable and logically separated into **upper and lower halves**, but readability could improve with:

- clearer variable names
- comments explaining the pattern logic
- modular structure (e.g., using a function).

3. Edge Cases

- `n = 1` works correctly and prints a single `*`.
- `n = 0` produces no output, which should instead trigger **input validation**.

4. Nested Loops Requirement The solution correctly uses **nested loops** and avoids string shortcuts such as `"*" * n`, satisfying the assignment requirement.

5. Time Complexity The algorithm runs in **O(n²)** time because each row prints up to `n` characters and there are about `2n` rows. This is expected for pattern-printing problems.

PART D: Bonus Challenge

```
# transaction_analyzer.py

transactions = []
total_credit = 0
total_debit = 0
count = 0
highest_transaction = 0

category_spending = {}

while True:

    entry = input("Enter transaction amount or 'done': ")

    if entry.lower() == "done":
        break

    amount = float(entry)
    t_type = input("Type (credit/debit): ").lower()
    category = input("Category (food/travel/bills/other): ").lower()

    transactions.append(amount)
    count += 1

    if amount > highest_transaction:
        highest_transaction = amount

    # Credit / Debit tracking
    if t_type == "credit":
        total_credit += amount
    elif t_type == "debit":
        total_debit += amount

        # Category tracking
        if category not in category_spending:
            category_spending[category] = 0
        category_spending[category] += amount

    # High-value flag
    if amount > 10000:
        print("⚠ High value transaction detected!")

print("\n----- LAST 10 TRANSACTIONS BAR CHART -----")

last_transactions = transactions[-10:]

for amt in last_transactions:
    stars = int(amt // 1000)

    print(f"{amt:>8} | ", end="")

    for i in range(stars):
        print("*", end="")

    print()

# Summary statistics
net_balance = total_credit - total_debit
average = sum(transactions) / count if count > 0 else 0
```

```
print("\n----- SUMMARY -----")
print("Total transactions:", count)
print("Total credits:", total_credit)
print("Total debits:", total_debit)
print("Net balance:", net_balance)
print("Highest transaction:", highest_transaction)
print("Average amount:", round(average, 2))

print("\n----- CATEGORY SPENDING -----")

for cat in category_spending:
    print(cat, ":", category_spending[cat])
```

```
Enter transaction amount or 'done': 200
Type (credit/debit): 15000
Category (food/travel/bills/other): 2400
Enter transaction amount or 'done': done

----- LAST 10 TRANSACTIONS BAR CHART -----
   200.0 |

----- SUMMARY -----
Total transactions: 1
Total credits: 0
Total debits: 0
Net balance: 0
Highest transaction: 200.0
Average amount: 200.0

----- CATEGORY SPENDING -----
```

Start coding or generate with AI.